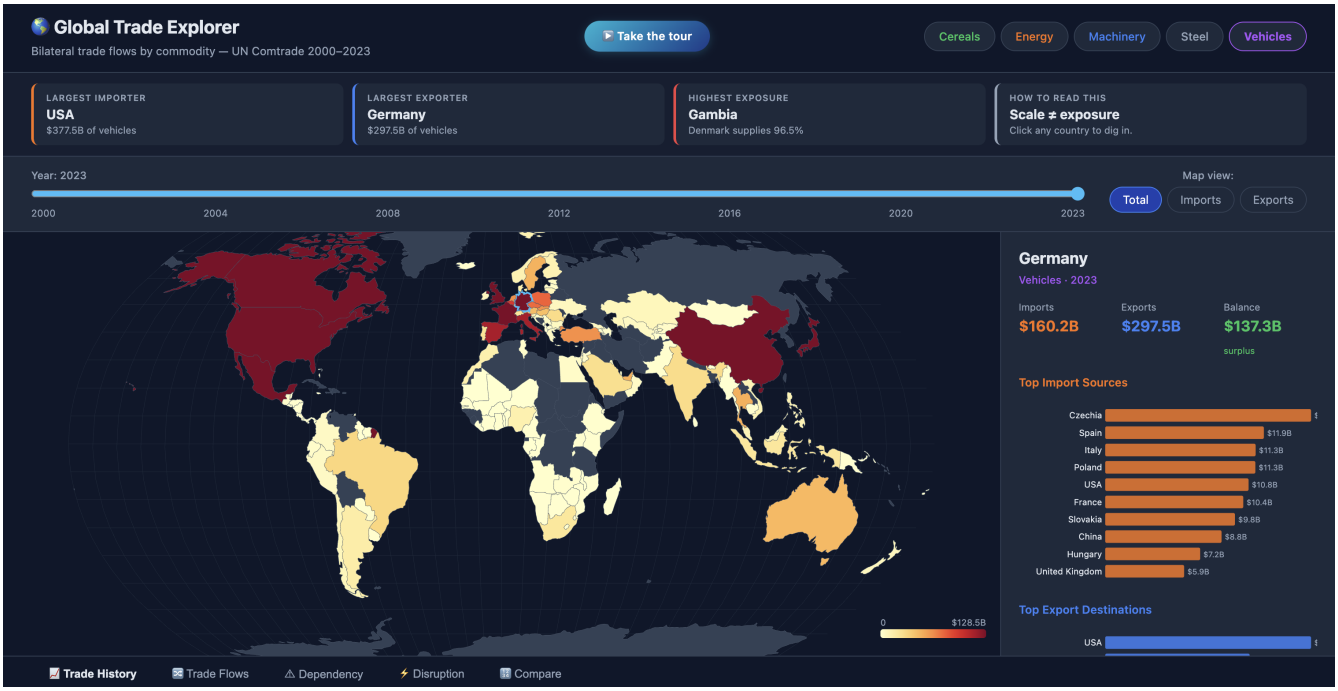
Global Trade Explorer

Dependencies, Flows & Disruptions

COM-480 Data Visualization • Process Book

Team **Vizion** • Charbel Raffoul, Nicolas Khamis, Elyas El Khaldi, Kevin Drakebli • EPFL, Spring 2026



The final D3.js dashboard: choropleth entry point, an always-on storytelling strip, year slider, commodity selector, and a click-to-drill country panel. ***vizion-i8ez.onrender.com***

About this document. A reflective process book describing how the Global Trade Explorer was built across three milestones: from framing trade dependency as a structural question, through an exploratory analysis and a first Dash prototype, to a full rewrite in **D3.js** with a what-if disruption simulator and a geopolitical bloc model. We cover the path we took, the design decisions we made, the challenges we hit, and how the work was split across the team.

5	**24**	**393 k**	**~3 500**	**11**
commodities	years (2000–2023)	Comtrade rows	lines of code	interactive views

1 Origin: framing the right question

Global trade is the invisible scaffolding of daily life: the wheat in bread, the steel in buildings, the gas that heats homes. Yet most existing trade dashboards stop at *totals*: how much was traded, by whom, in what year. We found that framing unsatisfying. The front-page trade stories of the 2020s, the 2022 European energy shock, the 2020 semiconductor shortage, the Red Sea disruptions, are not about totals. They are about *structure*: how concentrated each country's supply chain is, how exposed it is to a single partner, and what happens when a corridor breaks.

Our goal became: build an interactive web dashboard that makes the *structure* of global trade legible, not just its volume. A user picks a commodity family (energy, cereals, steel, machinery, vehicles), and the dashboard answers three questions:

1. **Who trades what with whom?** (choropleth, Sankey flows)
2. **How dependent is each country, and on whom?** (top-partner share, top-3 share, dependency race)
3. **How has this structure shifted, and what happens if it breaks?** (top movers, disruption simulator, trade blocs, trade gravity)

Existing platforms (OEC, World Bank, UN Comtrade dashboards) answer (1) well and (2) only partially; none address (3) interactively. That gap defined the project, and the third question became the centre of gravity for our final milestone.

1.1 Audience and design principle

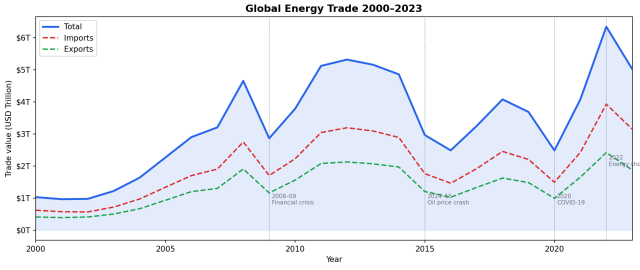
The target audience is students, researchers, journalists, and engaged citizens with an interest in global economics or supply-chain resilience. They are not necessarily economists. From this we extracted the central design principle, drawn from the *Interaction* lecture:

Design decision

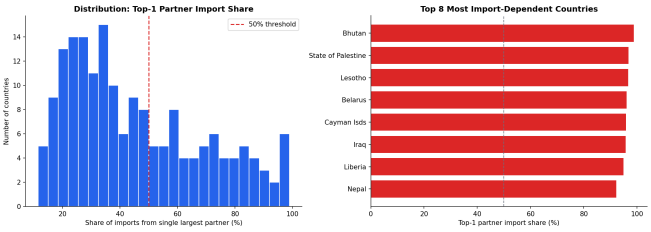
Progressive disclosure. A first-time visitor must understand the dashboard in ten seconds. A power user must be able to drill into dependency curves, animate them over decades, simulate a supply shock, and re-draw the world as competing trade blocs. We keep the front page simple (one map, one slider, one commodity selector, four headline cards) and push every advanced view behind a click or a scroll. A 90-second guided tour gives newcomers a narrated path through the whole story.

2 The data: UN Comtrade

We use the **UN Comtrade database**, which publishes annual bilateral trade flows at the HS (Harmonized System) product level. Our window is 2000–2023 and our scope is five HS chapters chosen for their geopolitical salience: **HS 27** mineral fuels, **HS 10** cereals, **HS 72** iron & steel, **HS 84** machinery, **HS 87** vehicles. Energy (HS 27) was the natural pilot; the other four were added once the pipeline generalised.



(a) Global energy trade, 2000–2023. The series grows roughly 6× (from \$0.7 T to \$4.0 T) with four visible shocks: the 2008 financial crisis (−35%), the 2014–16 oil crash, the 2020 COVID dip (−25%), and the 2022 geopolitical spike.



(b) Trade concentration analysis. A long tail of small economies sources > 50% of their energy imports from a single partner. This skewed distribution motivated the Dependency panel's diversified / moderate / concentrated bands.

The EDA notebook confirmed three things that shaped every downstream choice: **(a)** trade values span five orders of magnitude in USD (forcing a log scale on the map); **(b)** dependency is heavily skewed; **(c)** four temporal shocks structure the entire 24-year window, turning "show change over time" into a richer story than expected.

2.1 Findings: what the data revealed

Before building any visualization, the EDA already surfaced concrete claims that later became the dashboard's stories:

Finding

Energy trade more than sextupled between 2000 and 2022, from \$0.7 T to \$4.0 T, with four discrete shocks (2008, 2014–16, 2020, 2022) visible in both imports and exports.

Finding

Top importers are the largest economies (USA, China, Japan, Germany, South Korea, India); **top exporters are a different set** (Russia, Saudi Arabia, Canada, Norway, UAE). Trade is structurally a producer-vs-consumer split.

Finding

Smaller economies are the most dependent. The most-import-dependent ranking is dominated by Bhutan, Palestine, Lesotho, Belarus, and Liberia, each sourcing $\geq 80\%$ of their energy from a single partner. Large economies are well diversified.

Finding

Three corridors dominate the energy world. Canada→USA is the single largest globally; Russia→EU was second until 2022; the Persian Gulf→East Asia spine connects the Middle East to Japan, Korea, China.

3 The path: from sketch to prototype to rewrite

3.1 Milestone 1: framing and EDA

Milestone 1 was about understanding what we had. We acquired the Comtrade data via API, built a preprocessing pipeline (`src/preprocess.py`, `data_loader.py`, `utils.py`), and produced an exploratory notebook documenting coverage, shocks, and dependency structure. The deliverable was a static choropleth and a written argument that *structure*, not totals, was the interesting question. We did not produce hand-drawn sketches; the EDA figures themselves *were* the visual plan, and every later view is the direct, interactive expansion of one of them (see the mapping below).

3.2 Milestone 2: the Dash prototype

For the prototype we chose **Dash (Plotly under the hood)**: it gave us interactivity on top of Plotly's mature choropleth and Sankey primitives in Python, reusing the same pandas code that powered the EDA. By M2 we had a working multi-commodity dashboard with a choropleth, a click-to-drill side panel, and tabbed Trade History / Sankey / Dependency / Compare views, plus Top Movers and an animated Dependency Race.

Design decision

Prototype fast in Dash, then commit to D3. Dash let us validate the whole concept in days rather than weeks. But it carried two costs that pushed us to rewrite for M3: Plotly's opinionated rendering limited fine visual control, and loading five pandas datasets at runtime repeatedly crashed Render's 512 MB free tier (out-of-memory). The prototype proved the *idea*; the rewrite delivered the *product*.

3.3 Milestone 3: the rewrite to D3.js

For the final milestone we rebuilt the entire frontend in **pure D3.js**, served by a thin **Flask** backend. This was the single biggest decision of the project and it unlocked everything that makes the final product distinctive.

Design decision

Pre-compute everything; serve static JSON; render with D3. A build script (`build_data.py`) turns the processed CSVs into compact per-commodity JSON blobs once, at build time. Flask just serves them; *no pandas runs at request time*. This eliminated the OOM crashes, cut the cold start dramatically, and gave us full control over every SVG element, which the three signature M3 views (disruption, blocs, gravity) absolutely required.

3.4 From M1 findings to final views

Each static EDA observation set the agenda for an interactive view. The mapping makes the "expand the sketch" path explicit:

M1 finding (static EDA)		Final interactive view (D3)
6× growth with four shocks	→	**Trade History** with shock context, per country
Top-1 share is heavily skewed	→	**Dependency** panel with diversified/moderate/concentrated bands
Most-dependent-country ranking	→	**Dependency Race** animated across 24 years
Producer-vs-consumer split	→	**Compare** and **Top Traders**
Three corridors dominate	→	**Sankey flows** on demand
"what if a corridor breaks?"	→	**Disruption**, **Trade Blocs**, **Trade Gravity** (new in M3)

4 The final dashboard

4.1 Core exploration

The entry point is a log-scaled choropleth with a year slider (2000–2023), a commodity selector, and an import/export/total toggle. Clicking any country opens a side panel with its imports, exports, balance, and top-partner bars, plus four tabs: **Trade History** (imports vs. exports over time), **Trade Flows** (a Sankey of top bilateral partners), **Dependency** (top-1 and top-3 partner share over time, with risk bands), and **Compare** (two countries side by side). Two always-visible sections below, **Top Traders** and the animated **Dependency Race**, give global context.

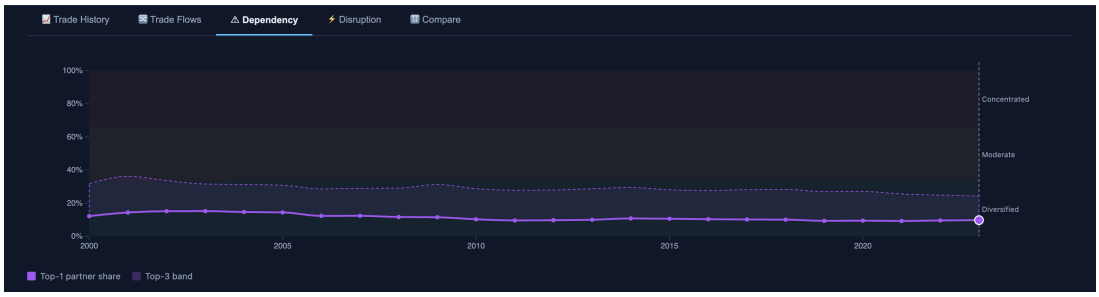
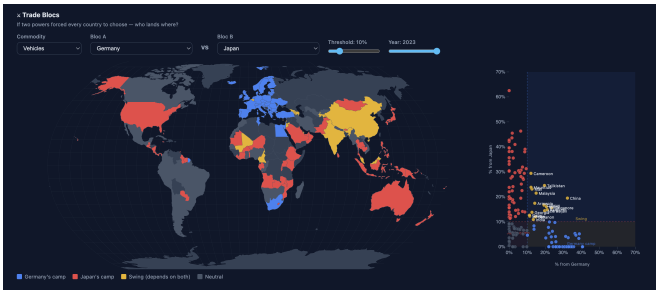


Figure 2: Dependency panel. Two intuitive lines (top-1 and top-3 partner share) tracked over 24 years, with diversified / moderate / concentrated bands so a non-economist can read concentration risk at a glance. *Lecture connections:* Rankings; Perception (banded ranges); Time-series.

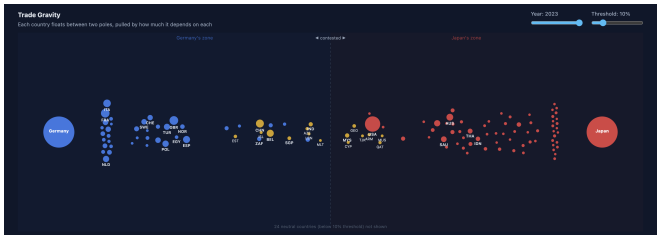
4.2 The signature M3 features

These three views are what set the project apart from existing trade dashboards. They answer the third question: *what happens when the world stops being neutral?*

Figure 3: Disruption Simulator. Remove a country from the network and the map recolours by who is hit. A *supply-shock* mode (remove it as an exporter) shows who loses a source; a *demand-shock* mode (remove it as an importer) shows who loses a customer, a completely different set of countries. This two-sided framing is the feature's core insight.



(a) Trade Blocs. Pick two anchor economies; every country is coloured by which camp it falls into (blue / red) or whether it is a *swing* state depending on both (yellow). An adjustable threshold and year redraw the geopolitical map of alignment.



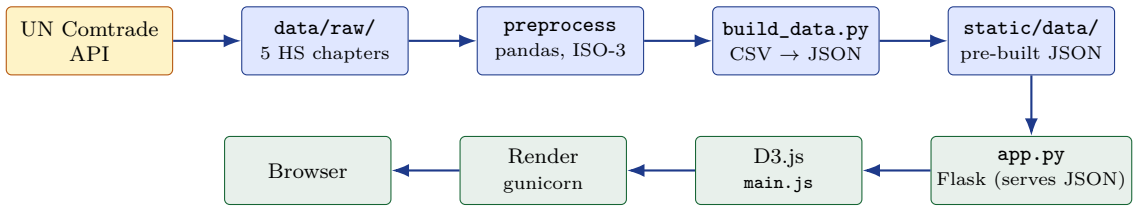
(b) Trade Gravity. The same data as a D3 force simulation: each country is pulled toward two poles by how much it depends on each, sized by import volume. Distance from the centre measures how hard its choice would be.

4.3 Storytelling layer

Two complementary layers turn the tool into a narrative, addressing the rubric's "tell a data story" requirement directly in the product:

- **Always-on takeaway strip.** Four headline cards above the map (largest importer, largest exporter, highest single-partner exposure, and a "how to read this" guide) update on every commodity or year change, so every view has an instant punchline.
- **90-second guided tour.** A "Take the tour" button launches a seven-step narrated walkthrough that mirrors our screencast: vehicles 2023 → Germany → dependency → a Germany supply shock → the Germany-vs-Japan bloc map → trade gravity → "now explore on your own."

5 Architecture and implementation



Build-time artefacts (amber/navy) run once; the runtime path (green) is pure static JSON served by Flask to a D3 frontend. No pandas at request time, which is what fixed the free-tier memory crashes.

Layer	What we used
Ingestion	UN Comtrade API via `comtradeapicall`; per-HS-chapter downloaders
Preprocessing	pandas, NumPy; ISO-3 normalisation; `build_data.py` emits compact JSON
Backend	**Flask**, serving pre-built JSON; `server` object run under gunicorn
Frontend	**D3.js v7** (+ d3-sankey, topojson); one `main.js`, no build step
Views	choropleth, Sankey, line/bar charts, animated race, force simulation, dual-mode disruption map
Hosting	Render free tier (EU-FRANKFURT); auto-deploy on push; vizion-i8ez.onrender.com

6 Challenges and design decisions

Challenge

Render's free tier (512 MB) kept killing the Dash app. Loading five pandas datasets at request time blew the memory cap and crashed the worker.

Design decision

Move all computation to build time. `build_data.py` pre-computes every view's data into static JSON. The runtime process holds almost nothing in memory, so it fits comfortably and starts fast. This is also why the rewrite to D3 was worth it, not just aesthetics.

Challenge

The disruption question has two valid answers. "Remove Germany" could mean it stops selling *or* stops buying, and these hit opposite sets of countries.

Design decision

Ship both, behind one toggle. Supply-shock and demand-shock modes reuse the same map but query export-side vs import-side dependence. The toggle makes the symmetry of trade visible, which no comparable dashboard does.

Challenge

About 10% of partner rows have no ISO-3 code (inconsistent spellings, aggregates like "World"), and the topojson map uses numeric IDs.

Design decision

A manual override table plus a numeric-to-ISO3 lookup in the pipeline and the frontend; aggregates are filtered before any bilateral analysis so the map never tries to colour "World".

Challenge

Re-export inflation (Netherlands, Belgium, Singapore) makes naive top-partner counts misleading.

Design decision

Use direct importer-reported shares as the headline metric and document the caveat, rather than attempting full re-export attribution.

Challenge

The raw energy file is 122 MB, above GitHub's 100 MB limit.

Design decision

Whitelist-based `.gitignore`: commit only the small derived files the app needs; the large raw aggregate stays out, and a fresh clone still runs.

7 Case study: the Trade Blocs view

The Trade Blocs view is the project's most original idea and the hardest to get right. This is its design history.

The question. *If the world fragmented into two camps anchored by two economies, where would every other country land?* It is a what-if, not a description, which is exactly the kind of question existing dashboards cannot ask.

Alternatives we considered.

- *Two separate choropleths* (share-from-A, share-from-B). Rejected: forces the eye to compare two maps; the alignment story disappears.
- *A single "A minus B" diverging map.* Rejected: hides the swing states, which are the most interesting countries.
- *Categorical camps (blue / red / yellow swing) with an adjustable threshold, shown as a map* and *a scatter.* **Chosen.** The map gives geography; the scatter (share-from-A vs share-from-B) gives the exact position; the threshold lets the user test how robust an alignment is.

Design decision

Show the swing states, and let the user move the line. The yellow swing countries, those depending meaningfully on both anchors, are the punchline: they are the ones with the most to lose from fragmentation. The threshold slider turns a static claim into an interactive argument. The **Trade Gravity** force simulation then re-expresses the same data as physics, making "how hard is the choice?" visceral rather than categorical.

8 Reflection

8.1 What we learned

Prototype throwaway, then commit. The Dash prototype was the right call for validating the concept quickly, and rewriting in D3 was the right call for the final product. Trying to do either alone would have been worse: pure-D3 from day one would have been slow to iterate; staying in Dash would have capped both performance and visual ambition.

The structural questions were the project. We spent early effort polishing the choropleth, but the dependency, disruption, and bloc views are what make the dashboard worth using. In hindsight we would have reached the what-if features sooner.

8.2 Known limitations

- Values are nominal USD; no inflation adjustment, so magnitudes inflate over time.
- Mirror-trade asymmetry is not reconciled; we prefer importer-reported values.
- Re-export inflation is documented but not corrected.
- 2023 coverage is partial; Render's free tier cold-starts in 30–60 s.

9 Peer assessment

Area	Tasks	Owner
Data acquisition & EDA	Comtrade API, preprocessing pipeline, quality & exploratory notebooks	Nicolas Khamis
Multi-commodity pipeline	HS 10/72/84/87 downloads, schema unification, `build_data.py` JSON	Nicolas Khamis
D3 rewrite & core views	Flask backend, choropleth, side panel, Sankey, history, dependency	Charbel Raffoul
Signature M3 features	Disruption simulator (supply/demand), Trade Blocs, Trade Gravity	Charbel Raffoul
Storytelling layer	Takeaway strip concept & copy, missing-data handling, runnable workflow	Elyas El Khaldi
Guided tour	7-step scripted tour wiring the dashboard to the screencast	Nicolas & Elyas
Deployment & repo	Render config, gunicorn, two-remote workflow, cleanup	Nicolas Khamis
Screencast	Script, recording, voiceover, editing (2-min)	Kevin Drakebli
Reports & process book	M1 notes, M2 report, this process book	Nicolas & Charbel

Thank you for reading. Live dashboard: vizion-i8ez.onrender.com. Code: github.com/com-480-data-visualization/Vizion.